Universidad Técnica Estatal de Quevedo
Facultad de ciencias de la computacion Ingeniería de Software
Aplicaciones Distribuidas (ISR-701)

TA-IND-01

Análisis crítico comparativo de un sistema distribuido

Melanie Michelle Muñoz Vera
https://github.com/Melanir/isr701-ta-ind-01

Borrador de estructura

Resumen

Este informe analiza Apache Cassandra como sistema distribuido y lo compara con MongoDB para identificar cómo sus decisiones de replicación afectan la disponibilidad y la consistencia. Se configuraron tres nodos de Cassandra en Docker, en una sola computadora, con factor de replicación tres. Se insertó una fila con el nivel `ALL` y se consultó antes, durante y después de detener dos contenedores. Con todos los nodos activos, `ONE` y `QUORUM` devolvieron la fila. Durante la caída, `ONE` siguió respondiendo y `QUORUM` falló porque exigía dos réplicas accesibles. Tras recuperar los nodos, `QUORUM` volvió a funcionar. La comparación documental muestra que MongoDB utiliza una mayoría de votantes para mantener o elegir un primario, mientras Cassandra exige un número de respuestas según el nivel de cada operación. Los resultados respaldan la elección de `ONE` para lecturas que toleran datos desactualizados. La prueba mide si la consulta puede completarse. Evaluar la latencia, las escrituras concurrentes y las particiones de red requeriría experimentos adicionales.

1. Introducción

Apache Cassandra es una base de datos NoSQL de columnas anchas. La usan equipos que necesitan distribuir grandes volúmenes de datos entre varios servidores y mantener el servicio disponible cuando falla un nodo. Un ejemplo es Discord. Su equipo de ingeniería documentó que, a principios de 2022, almacenaba billones de mensajes en un clúster de 177 nodos Cassandra. El crecimiento también trajo problemas de latencia y mantenimiento que motivaron su migración posterior a ScyllaDB [1], [2].

Este informe analiza cómo el nivel de consistencia afecta la posibilidad de leer un dato cuando se pierden réplicas. La prueba utiliza tres nodos Cassandra en Docker y compara la misma lectura con `ONE` y `QUORUM` antes, durante y después de detener dos nodos. Como contexto, se toman estudios que comparan configuraciones de consistencia de Cassandra, MongoDB y Redis en la nube [3], [4]. La comparación arquitectónica con MongoDB se apoya en su documentación sobre conjuntos de réplicas [5]. La práctica local se limita a Cassandra y observa si la consulta puede completarse.

2. Arquitectura de Cassandra

Cassandra funciona como un clúster de nodos pares. El nodo que recibe una solicitud del cliente actúa como coordinador de esa operación y se comunica con las réplicas que corresponden al dato. Los nodos mantienen información sobre la membresía y el estado del clúster mediante el protocolo *gossip* [1].

Para ubicar los datos, Cassandra calcula un hash de la clave de partición y lo usa como token para determinar qué nodos almacenan la partición. En la tabla `prueba`, la clave de partición es `id`. El espacio de claves `ta_ind_01` usa `NetworkTopologyStrategy` con un factor de replicación de tres en `datacenter1`. Así, los tres nodos guardan una copia de la partición consultada. El nivel de consistencia indica cuántas réplicas deben responder para completar una operación. En esta configuración, una lectura con `ONE` requiere una respuesta y con `QUORUM` requiere dos [1].

La Figura 1 muestra el recorrido de la consulta en el montaje de Cassandra 5.0.9. Desde PowerShell se ejecuta `cqlsh` dentro del contenedor del nodo 1, mediante `docker exec`. El nodo 1 coordina la consulta y también almacena una réplica. Los nodos 2 y 3 guardan las otras dos. Los tres contenedores se ejecutan en una sola computadora, por lo que el montaje permite observar fallos de nodos individuales dentro del mismo equipo.

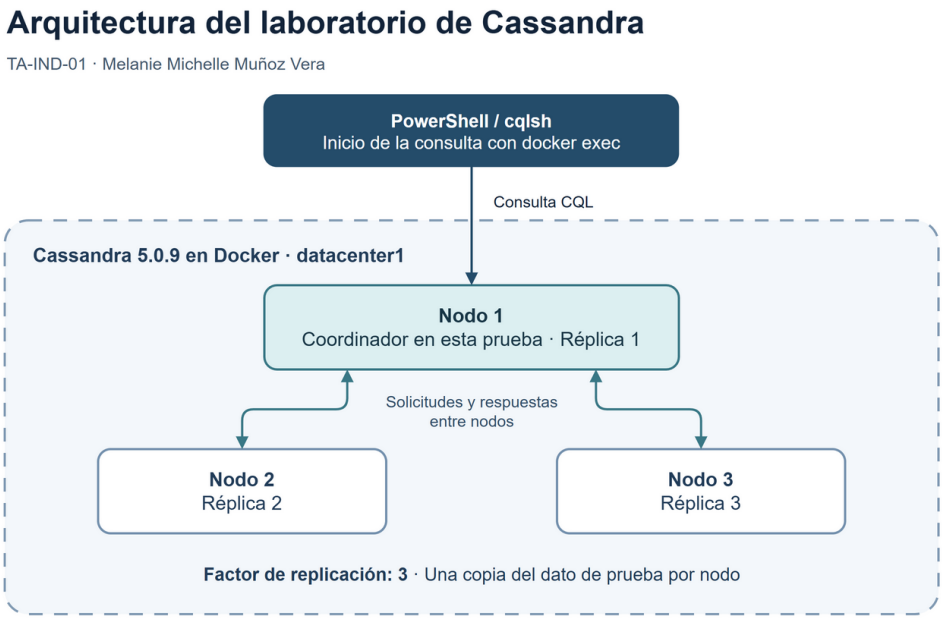


Figura 1: Recorrido de una consulta CQL en el laboratorio. Diagrama elaborado a partir del bosquejo del Anexo A.

3. Análisis con los conceptos de la Unidad 1

3.1. Transparencias de distribución

Acceso y ubicación. En la práctica usé la misma consulta CQL para leer la fila con tres nodos y con uno. El coordinador localiza las réplicas mediante la clave de partición. La dirección del nodo al que se conecta el cliente sí forma parte de la configuración [1].

Migración y reubicación. Cassandra puede redistribuir rangos de datos al cambiar la composición del clúster. La consulta conserva la misma clave, pero un cambio de dirección del nodo al que está conectada la sesión puede requerir reconexión. La prueba mantuvo fijo el nodo coordinador y no evaluó movimientos de datos con solicitudes en curso [1].

Replicación, concurrencia y fallos. El coordinador decide qué réplicas contactar según la operación. El cliente sí elige el nivel de consistencia. En escrituras ordinarias que compiten por el mismo dato, Cassandra resuelve el conflicto usando marcas de tiempo [1]. En la prueba, la caída fue transparente para la lectura con `ONE`, que conservó el mismo resultado. Con `QUORUM`, el error hizo visible la falta de réplicas.

3.2. CAP y PACELC

CAP. Si una partición de red impide que los nodos se comuniquen, el teorema CAP establece que el sistema no puede garantizar a la vez la consistencia linealizable y una respuesta exitosa a toda solicitud que llegue a un nodo operativo [6]. La prueba de Cassandra permite examinar una consecuencia relacionada. Se detuvieron dos contenedores, de modo que el coordinador solo tenía una réplica disponible. Esa réplica devolvió la fila con `ONE`. Con `QUORUM`, el servidor devolvió `Unavailable` porque necesitaba dos réplicas y solo tenía una activa. El resultado muestra cómo el mínimo de respuestas afecta la disponibilidad de una lectura.

Cassandra permite elegir el nivel de consistencia por operación [1]. Con `ONE`, en otra ejecución, una lectura podría obtener un valor desactualizado si la réplica consultada aún no recibió una escritura reciente. En esta prueba, la fila se insertó con `ALL` antes de detener los nodos y se leyó sin modificarla. La pérdida de acceso a dos réplicas sirve para ilustrar el dilema de CAP. Comprobar

el comportamiento ante una partición real requeriría mantener los nodos activos y bloquear la comunicación entre ellos. Evaluar la consistencia linealizable requeriría además observar el orden de lecturas y escrituras.

PACELC. El modelo de Abadi plantea una elección entre disponibilidad y consistencia durante una partición (*PAC*), y entre latencia y consistencia cuando la red funciona con normalidad (*ELC*) [7]. Con los tres nodos activos, ambas lecturas devolvieron la fila. `QUORUM` debía esperar dos respuestas para completar la consulta, mientras que a `ONE` le bastaba una. Esto permite plantear que esperar una segunda respuesta puede aumentar la latencia, sobre todo si una réplica tarda en responder. Las capturas registran el éxito o fallo de la consulta y la hora de cada etapa. Medir la latencia requeriría cronometrar consultas repetidas bajo las mismas condiciones.

Como contraste, Redžepagić et al. midieron la latencia de *escritura* de Cassandra en Kubernetes. En la Tabla 22 de su estudio, el promedio pasó de 1,23 ms con `ONE` a 3,16 ms con `QUORUM` [8]. Esas cifras corresponden a la carga y al entorno de ese estudio. En el laboratorio de este informe, la comparación se limitó a la disponibilidad de una lectura.

3.3. Modelos de comunicación

El sistema combina un modelo cliente-servidor entre `cqlsh` y Cassandra con un modelo entre pares dentro del clúster. El cliente envía solicitudes CQL mediante el protocolo binario nativo sobre TCP y recibe un resultado o un error [9]. En el laboratorio, cada consulta de la terminal esperó su respuesta. Entre nodos, el coordinador intercambia mensajes con las réplicas y la operación espera las respuestas que exige su nivel de consistencia. El intercambio periódico de estado por *gossip* ocurre en segundo plano [1].

3.4. Tolerancia a fallos

La replicación permitió leer la fila aun con dos nodos detenidos. `nodetool status` mostró dos nodos en `DN` (caídos, en estado normal dentro del anillo) y uno en `UN` (activo, en estado normal). Con factor de replicación tres, el umbral de `ONE` permite tolerar la pérdida de hasta dos réplicas para leer un dato presente en la réplica accesible. `QUORUM` requiere dos réplicas accesibles y, por tanto, admite la pérdida de una [1]. La prueba observó los extremos de tres y una réplica activa.

Tras reiniciar los contenedores, los tres nodos volvieron a `UN` y la consulta con `QUORUM` volvió a devolver la fila. Se recuperó así la capacidad de completar esa lectura con el umbral solicitado. Para comprobar la sincronización de las réplicas durante la recuperación, haría falta escribir un dato mientras los otros nodos están detenidos y revisar si lo reciben al volver. Además, los tres contenedores comparten una computadora. Si esta se apaga, los tres nodos quedan fuera de servicio al mismo tiempo.

Incidente en producción. El equipo de Discord documentó que algunas particiones de mensajes concentraban tantas lecturas que subía la latencia de los nodos responsables. El efecto alcanzaba otras consultas porque las operaciones con cuórum dependían de respuestas de esas réplicas [2, sección «Our Cassandra Troubles»]. El caso muestra que una réplica activa también puede degradar el servicio si se satura.

4. Experimentación propia

El 12 de noviembre de 2026 se probó si Cassandra 5.0.9 podía leer una fila escrita previamente, antes, durante y después de detener dos de sus tres nodos. El clúster se ejecutó en Docker, con factor de replicación 3 en `datacenter1`. Las horas de las capturas corresponden a Ecuador continental (UTC -5). La carpeta `experimento/` contiene la configuración (`compose.yaml`, `preparar.cql`), la secuencia completa (`COMANDOS.md`) y las salidas registradas (`RESULTADOS.md`).

4.1. Preparación del clúster

Desde la carpeta `experimento`, el clúster se levantó en PowerShell y se cargó el esquema de la tabla `ta_ind_01.prueba`. El primer `nodetool status` mostró los tres nodos activos (Figura 2).

```
$docker = "$env:LOCALAPPDATA\Programs\DockerDesktop\resources\bin\docker.exe"
$nodo = 'ta-ind-cassandra-nodo1-1'
& $docker compose -f compose.yaml up -d --wait --wait-timeout 420
& $docker exec $nodo nodetool status
Get-Content -Raw preparar.cql | & $docker exec -i $nodo cqlsh
```

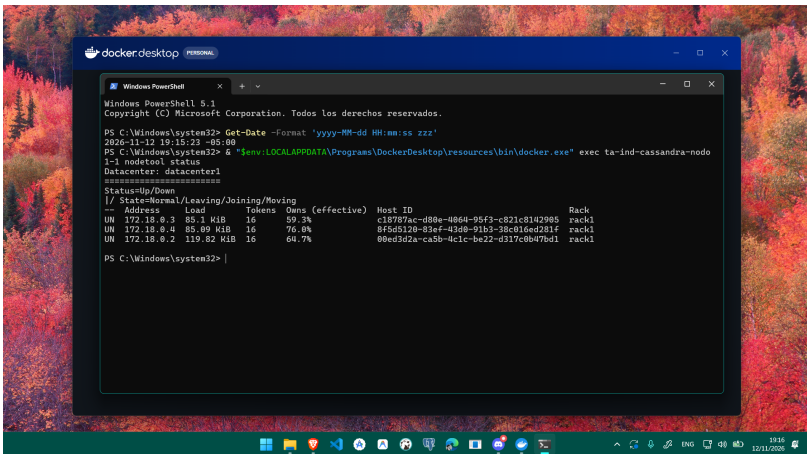


Figura 2: Estado inicial a las 19:15:23. Los tres nodos aparecen como `UN`.

4.2. Antes de la caída

La fila se insertó con `ALL`. Luego se consultó el mismo identificador con `ONE` y `QUORUM`. Ambas lecturas devolvieron la fila 1 con el mensaje «Prueba de Melanie» (Figura 3).

```
Get-Date -Format 'yyyy-MM-dd HH:mm:ss zzz'
@'
CONSISTENCY ALL;
INSERT INTO ta_ind_01.prueba (id, mensaje) VALUES (1, 'Prueba de Melanie');
CONSISTENCY ONE;
SELECT * FROM ta_ind_01.prueba WHERE id = 1;
CONSISTENCY QUORUM;
SELECT * FROM ta_ind_01.prueba WHERE id = 1;
'@ | & $docker exec -i $nodo cqlsh
```

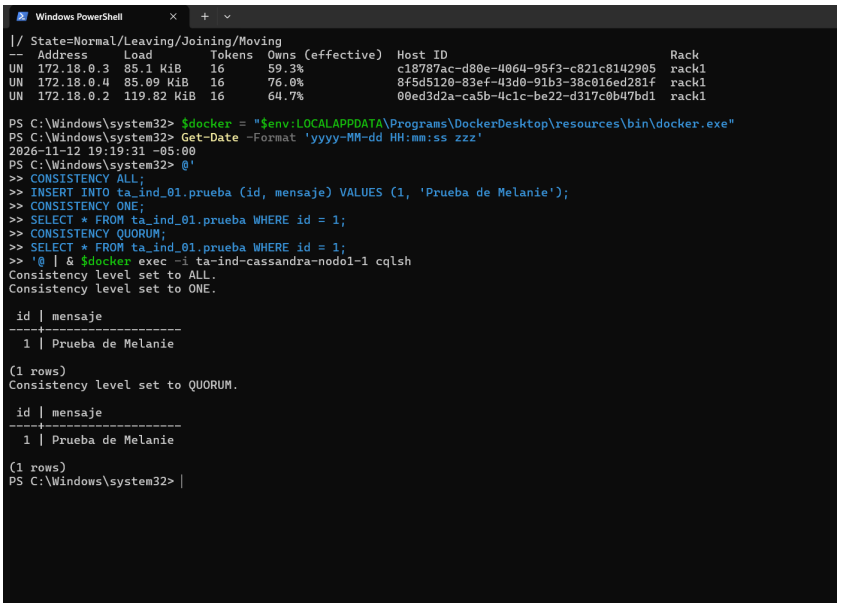


Figura 3: Escritura con `ALL` y lecturas con tres nodos activos a las 19:19:31.

4.3. Durante la caída

Se detuvieron los contenedores `nodo2` y `nodo3`. Cuando `nodetool status` mostró una fila `UN` y dos `DN`, se repitieron las lecturas desde el nodo 1.

```
& $docker compose -f compose.yaml stop nodo2 nodo3
Get-Date -Format 'yyyy-MM-dd HH:mm:ss zzz'
& $docker exec $nodo nodetool status
@'
CONSISTENCY ONE;
SELECT * FROM ta_ind_01.prueba WHERE id = 1;
CONSISTENCY QUORUM;
SELECT * FROM ta_ind_01.prueba WHERE id = 1;
'@ | & $docker exec -i $nodo cqlsh
```

`ONE` devolvió la fila 1. Con `QUORUM`, el servidor devolvió `Unavailable`. El cliente `cqlsh` mostró `NoHostAvailable`. El detalle del error indicó `required_replicas = 2` y `alive_replicas = 1` (Figura 4).

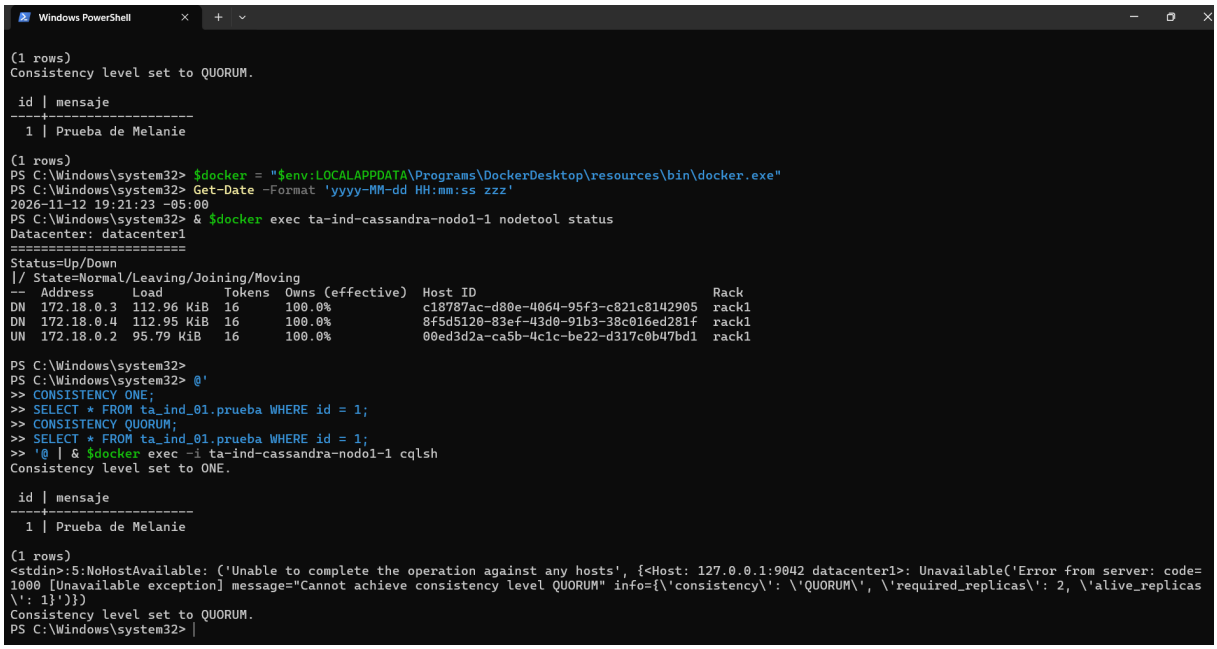


Figura 4: Lecturas con un nodo activo a las 19:21:23. El error de `QUORUM` aparece al final de la captura.

4.4. Después de la recuperación

Se levantaron de nuevo los contenedores detenidos y se repitió la lectura con `QUORUM`. Los tres nodos volvieron a aparecer como `UN` y la consulta devolvió la fila 1 (Figura 5).

```
& $docker compose -f compose.yaml up -d --wait --wait-timeout 360
Get-Date -Format 'yyyy-MM-dd HH:mm:ss zzz'
& $docker exec $nodo nodetool status
$consulta = 'CONSISTENCY QUORUM; SELECT * FROM ta_ind_01.prueba WHERE id = 1;'
& $docker exec $nodo cqlsh -e $consulta
```

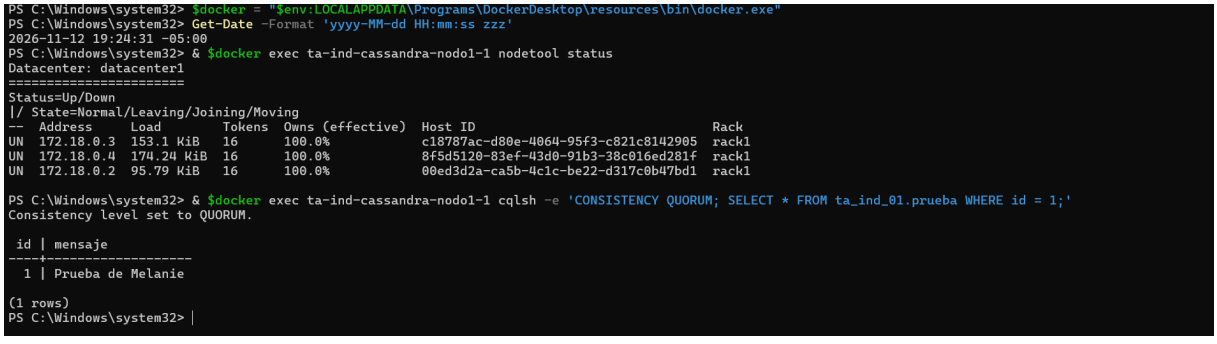


Figura 5: Lectura con `QUORUM` tras la recuperación a las 19:24:31.

Cuadro 1: Resultados de las lecturas de la fila 1 por etapa.

Etapa	Nodos activos	`ONE`	`QUORUM`
Antes de la caída	3	Devuelve la fila	Devuelve la fila
Durante la caída	1	Devuelve la fila	`Unavailable`
Tras la recuperación	3	Sin nueva consulta	Devuelve la fila

5. Ejemplos de mi entorno

5.1. Comunicación del tema por WhatsApp

Para esta actividad comuniqué mi tema en el grupo de WhatsApp de la asignatura. Envié un mensaje indicando que analizaría Apache Cassandra y lo compararía con MongoDB. En la conversación, mi mensaje aparece a las 17:46 y después se ven los temas que anunciaron otros compañeros a las 17:47 y 17:49.

Esta interacción ilustra la comunicación asíncrona entre personas. Cada integrante puede enviar su propuesta y los demás pueden consultarla y responder en otro momento. Las horas visibles permiten seguir el orden de los mensajes mostrados en mi pantalla. También se aprecia la transparencia de ubicación desde la perspectiva de uso. Para comunicar mi elección seleccioné el grupo, sin tener que conocer la ubicación física de los dispositivos de mis compañeros ni de los servidores del servicio.

5.2. Consulta de versiones del informe en GitHub

Al revisar el repositorio de esta actividad en GitHub, pude ver que el PDF del análisis de la Unidad 1 y el PDF de la experimentación quedaron en dos commits distintos, `4da2559` y `e601a2a`. El historial me permite identificar en qué versión se agregó cada PDF y consultar la versión del archivo que corresponde a ese commit.

Este ejemplo también muestra cómo se comunican dos partes de un sistema distribuido. Git permite trabajar sobre una copia local del repositorio y publicar cambios en el repositorio remoto [10]. Al consultar GitHub desde el navegador, se ve el estado publicado. Que una edición local todavía no aparezca en la web es una consecuencia de ese paso explícito de publicación. En la prueba de Cassandra, en cambio, el coordinador se comunica con las réplicas para atender la lectura en curso según el nivel solicitado. En ambos casos es necesario distinguir qué copias intervienen para interpretar lo que se ve en pantalla.

6. Análisis comparativo

La Tabla 2 compara Cassandra 5.0 con MongoDB 8.0 en un conjunto de réplicas. Ambos mantienen copias de los datos en varios nodos. La prueba de la Sección 4 aporta evidencia para Cassandra. Los escenarios de MongoDB se analizan a partir de su documentación.

Cuadro 2: Cassandra y MongoDB según seis criterios de la Unidad 1.

Criterio	**Cassandra 5.0**	**MongoDB 8.0**
Modelo de comunicación	Un nodo coordina la solicitud del cliente. Los nodos intercambian información de estado mediante *gossip* [1].	El primario recibe las escrituras del cliente. Los secundarios replican las operaciones de forma asíncrona [5].
Transparencia de replicación	El coordinador localiza las réplicas de la partición. El cliente puede elegir el nivel de consistencia [1].	El conjunto mantiene copias de los datos. El cliente puede configurar qué miembros usar para las lecturas [5].
Consistencia de lectura	Con tres réplicas, `ONE` pide una respuesta y `QUORUM` pide dos [1].	Una lectura `linearizable` en el primario puede reflejar, para un documento, todas las escrituras previas confirmadas por mayoría [11].
Partición de red (CAP)	Con acceso a una sola de tres réplicas, el coordinador puede leer con `ONE`. Con `QUORUM` necesita dos accesibles [1].	El primario del lado minoritario pierde su rol al detectar que dejó de comunicarse con la mayoría de votantes [12].
Latencia (PACELC)	En operación normal, esperar dos respuestas con `QUORUM` puede alargar una lectura frente a `ONE` [1], [7].	Una lectura `linearizable` puede esperar a que las escrituras concurrentes se propaguen a la mayoría [11].
Tolerancia y recuperación	Con tres réplicas, las lecturas con `QUORUM` pueden continuar si dos son accesibles. Otro nodo puede coordinar una nueva solicitud [1].	Si el primario falla, la mayoría de los votantes permite elegir un nuevo primario elegible. Durante la elección se interrumpen las escrituras [12].

6.1. Discusión

En el laboratorio, la fila ya estaba escrita con `ALL` cuando detuve dos contenedores. Con una réplica activa, `ONE` devolvió la fila y `QUORUM` produjo el error `Unavailable`. En Cassandra se cuentan respuestas de las réplicas de la partición para esa operación. En MongoDB se cuentan votos del conjunto para mantener o elegir un primario. Si solo queda accesible uno de tres votantes, el conjunto pierde el primario al detectar la falta de mayoría y se interrumpen las escrituras. Una lectura podría seguir disponible en un secundario si la configuración del cliente y el nivel de lectura lo permiten [5], [12].

La elección depende de lo que la aplicación necesite garantizar. Para una consulta que tolera datos desactualizados, `ONE` permite aprovechar la réplica accesible. Para una consulta de un documento que deba reflejar todas las escrituras previas confirmadas por mayoría, MongoDB ofrece `linearizable` bajo las condiciones indicadas en su manual [11]. El umbral de lectura `QUORUM` por sí solo cuenta respuestas. La garantía sobre el dato también depende del nivel de escritura y de las operaciones concurrentes [1].

La prueba local registró si la consulta pudo completarse. Para comparar tiempos habría que controlar la carga, la red y las garantías solicitadas. Ferreira et al. usaron `ONE` y `ALL` en Cassandra, y compararon lecturas `local` y `linearizable` en MongoDB en su propio entorno [4, sección «Database configurations»]. Sus mediciones corresponden a esas configuraciones.

7. Postura crítica

Fortalezas. Considero que la principal ventaja de Cassandra es poder ajustar el nivel de consistencia a lo que necesita cada operación. Al detener dos de los tres nodos, la lectura con `ONE` todavía devolvió la fila que había escrito antes de la caída. Esto resulta útil para consultas en las que importa seguir respondiendo y se puede tolerar que una réplica devuelva un dato desac-

tualizado. La arquitectura también permite que distintos nodos coordinen solicitudes sin elegir un primario para el clúster [1].

Limitaciones. Con dos nodos detenidos, `QUORUM` devolvió `Unavailable` porque quedaba una réplica activa y se necesitaban dos. La consulta volvió a funcionar cuando levanté los nodos. El resultado demuestra que la disponibilidad depende del nivel solicitado. Mi prueba se limita a una fila escrita con `ALL`, a lecturas sin escrituras concurrentes y a tres contenedores en una sola computadora. Por eso, el resultado sirve para evaluar la pérdida de nodos en esa configuración. Comparar la latencia o evaluar una partición de red exigiría pruebas específicas.

Cuándo elegiría otra opción. Para una aplicación en la que una decisión dependa del estado más reciente de un documento, como confirmar si un pedido ya fue cancelado, evitaría usar una lectura de Cassandra con `ONE`. Si el requisito es reflejar todas las escrituras previas confirmadas por mayoría sobre ese documento, consideraría MongoDB con lectura `linearizable` en el primario y escrituras confirmadas por mayoría [11]. Aceptaría que, si un conjunto de tres votantes pierde la mayoría, se interrumpan las escrituras hasta recuperar un primario [12]. En ese caso, me parece preferible que la operación espere o devuelva un error a que la aplicación actúe con un estado desactualizado.

8. Conclusiones

Con factor de replicación tres, una réplica activa bastó para leer con `ONE` una fila que ya estaba escrita. El mismo nodo devolvió un error con `QUORUM` porque ese nivel exigía dos respuestas. Al restaurar los tres nodos, la misma lectura con `QUORUM` volvió a completarse.

La disponibilidad debe analizarse por operación. En Cassandra, el nivel de consistencia determina el mínimo de respuestas para la consulta. En un conjunto de réplicas de MongoDB, la mayoría de votantes permite mantener o elegir al primario que recibe las escrituras [1], [12]. Para elegir entre ambos, primero definiría qué lecturas y escrituras deben seguir disponibles y cuánto retraso en la actualización de los datos puede aceptar la aplicación. El montaje local permitió observar el efecto de perder nodos. Quedan por evaluar la latencia y el comportamiento bajo una partición de red.

Declaración de uso de inteligencia artificial

Usé Gemini de Google para traducir fragmentos de las referencias en inglés. Usé Codex de OpenAI como guía para preparar y ejecutar la práctica de Cassandra y como apoyo para redactar, corregir y organizar el texto y el código LaTeX. La interpretación del experimento se comprobó con los comandos y las salidas de las cuatro capturas originales. Los datos bibliográficos se contrastaron con las publicaciones. Las versiones específicas de los modelos no quedaron registradas.

Referencias

[1] Apache Cassandra Project. «Dynamo.» Apache Cassandra Documentation, versión 5.0, visitado 12 de nov. de 2026. dirección: https://cassandra.apache.org/doc/stable/cassandra/architecture/dynamo.html

[2] B. Ingram. «How Discord Stores Trillions of Messages,» Discord, visitado 13 de nov. de 2026. dirección: https://discord.com/blog/how-discord-stores-trillions-of-messages

[3] S. Ferreira, J. Mendonça, B. Nogueira, W. Tiengo y E. Andrade, «Benchmarking Consistency Levels of Cloud-Distributed NoSQL Databases Using YCSB,» *IEEE Access*, vol. 13, págs. 63 428-63 438, 2025, doi: 10.1109/ACCESS.2025.3558923.

[4] S. Ferreira, J. Mendonça, B. Nogueira, W. Tiengo y E. Andrade, «Impacts of data consistency levels in cloud-based NoSQL for data-intensive applications,» *Journal of Cloud Computing*, vol. 13, n.º 1, 2024, art. 158, doi: 10.1186/s13677-024-00716-7.

[5] MongoDB. «Replication.» MongoDB Database Manual, versión 8.0, visitado 12 de nov. de 2026. dirección: https://www.mongodb.com/docs/v8.0/replication/

[6] S. Gilbert y N. Lynch, «Brewer's conjecture and the feasibility of consistent, available, partition-tolerant web services,» *ACM SIGACT News*, vol. 33, n.º 2, págs. 51-59, 2002, doi: 10.1145/564585.564601.

[7] D. J. Abadi, «Consistency Tradeoffs in Modern Distributed Database System Design: CAP Is Only Part of the Story,» *Computer*, vol. 45, n.º 2, págs. 37-42, 2012, doi: 10.1109/MC.2012.33.

[8] J. Redžepagić, A. Kapulica, N. Malešević y V. Dakić, «Empirical Performance and Operational Analysis of Monolithic and Distributed Database Architectures in Kubernetes Environments,» *Computers*, vol. 15, n.º 5, 2026, art. 282, doi: 10.3390/computers15050282.

[9] Apache Cassandra Project. «Native Protocol Versions.» Apache Cassandra Documentation, versión 5.0, visitado 14 de nov. de 2026. dirección: https://cassandra.apache.org/doc/stable/cassandra/reference/native-protocol.html

[10] GitHub. «Cloning a Repository.» GitHub Docs. dirección: https://docs.github.com/en/repositories/creating-and-managing-repositories/cloning-a-repository

[11] MongoDB. «Read Concern "linearizable".» MongoDB Database Manual, versión 8.0, visitado 14 de nov. de 2026. dirección: https://www.mongodb.com/docs/v8.0/reference/read-concern-linearizable/

[12] MongoDB. «Replica Set Elections.» MongoDB Database Manual, versión 8.0, visitado 14 de nov. de 2026. dirección: https://www.mongodb.com/docs/v8.0/core/replica-set-elections/

A. Esquema inicial a mano

El bosquejo sirvió de base para el diagrama de la arquitectura. En la versión digital se distingue `cqlsh` dentro del flujo de PowerShell y se mantiene el papel del nodo 1 como coordinador de la prueba.

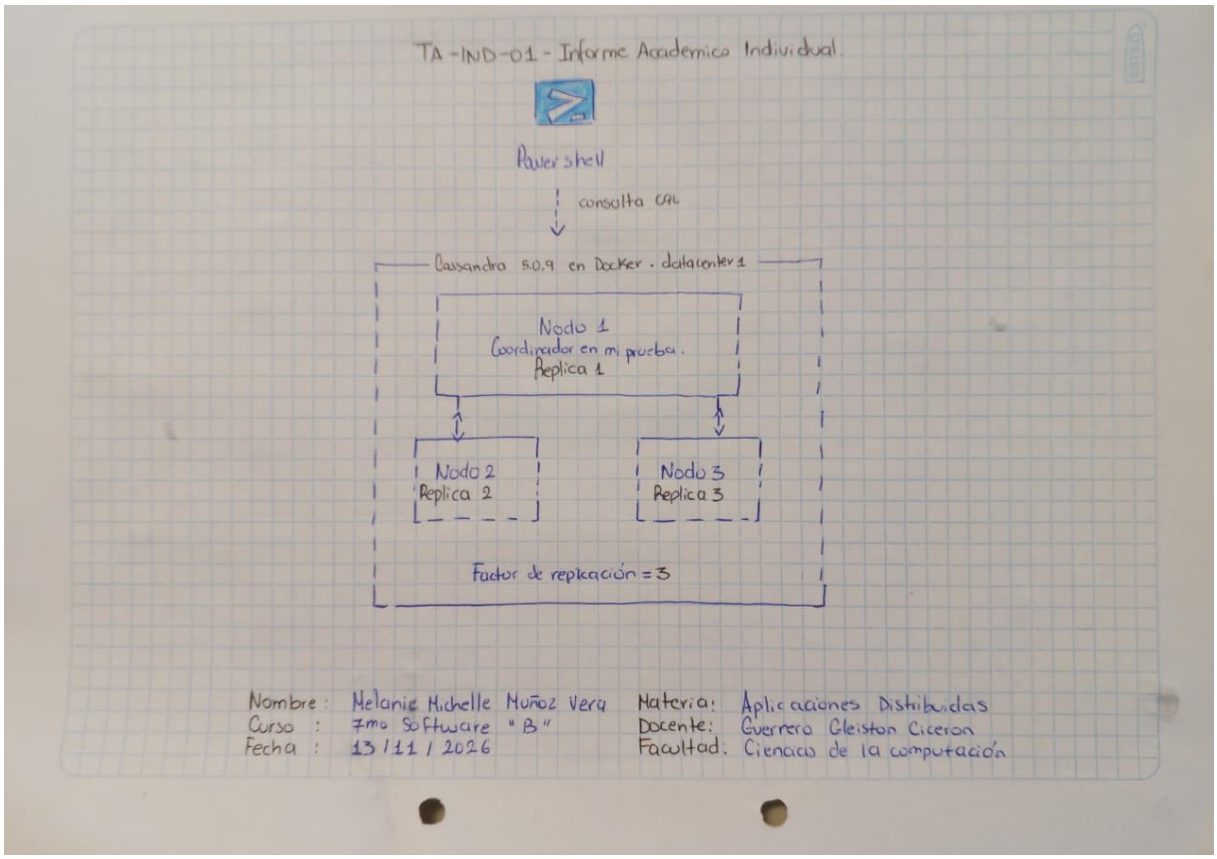


Figura 6: Fotografía del bosquejo original del clúster de Cassandra.

B. Lectura de las fuentes

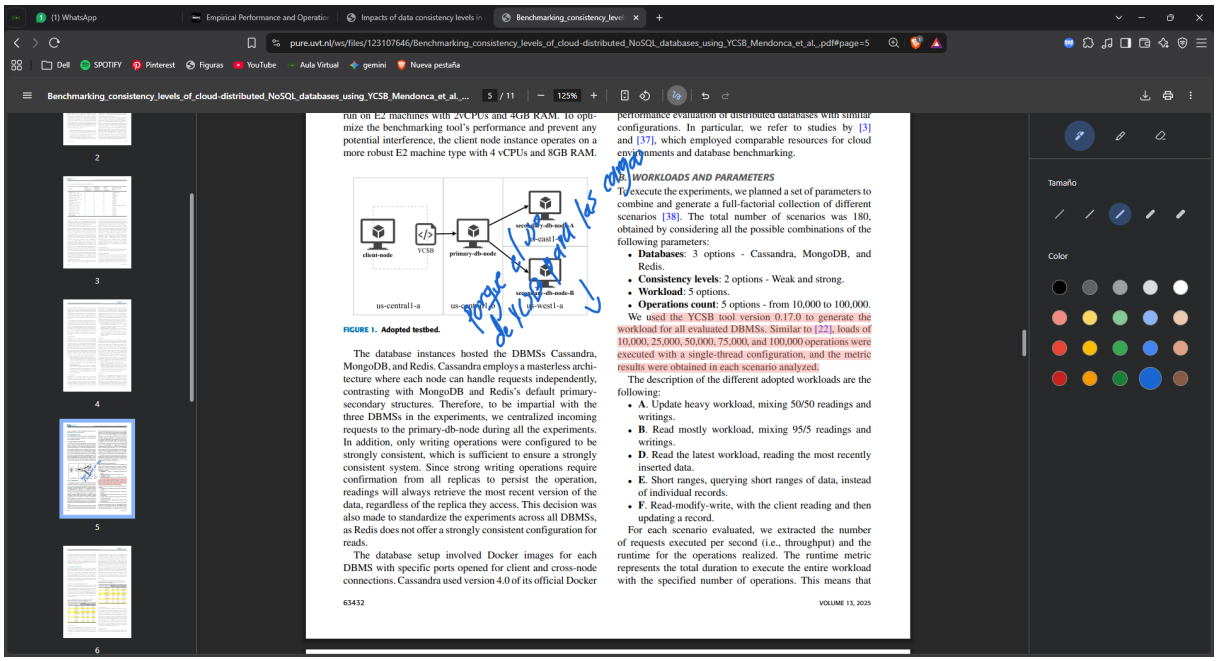


Figura 7: Ferreira et al., p. 63432. Sección sobre las cargas y los parámetros de YCSB [3].

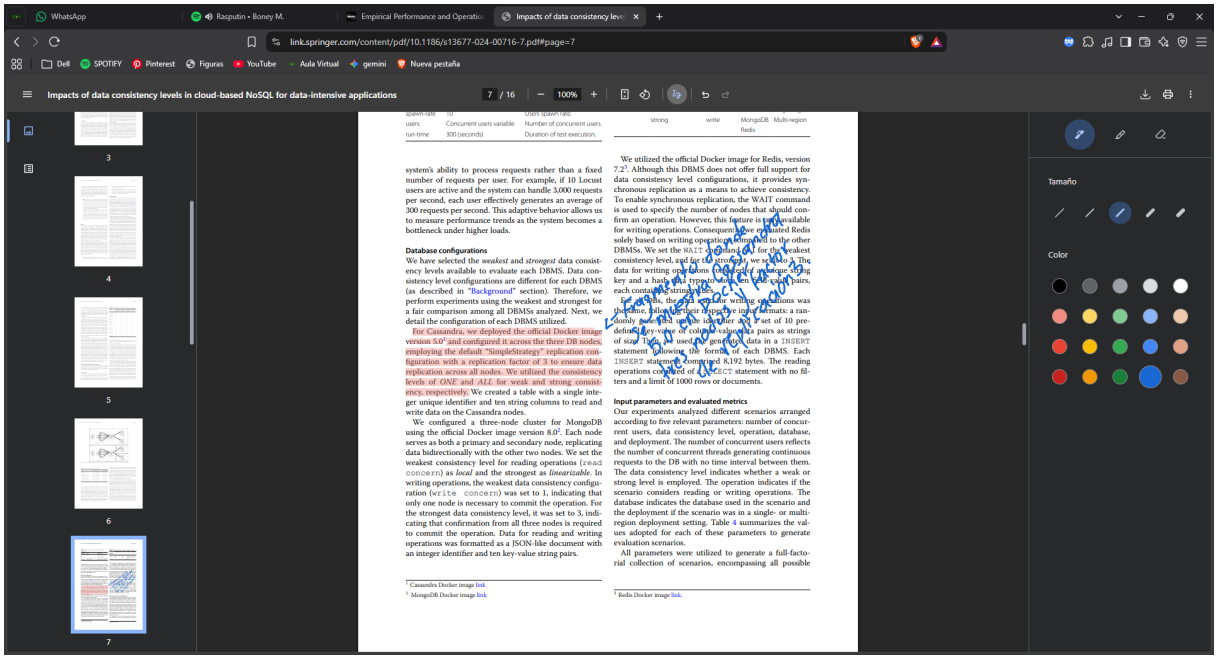


Figura 8: Ferreira et al., p. 7 del PDF. Configuración de Cassandra con tres nodos y niveles `ONE` y `ALL` [4].

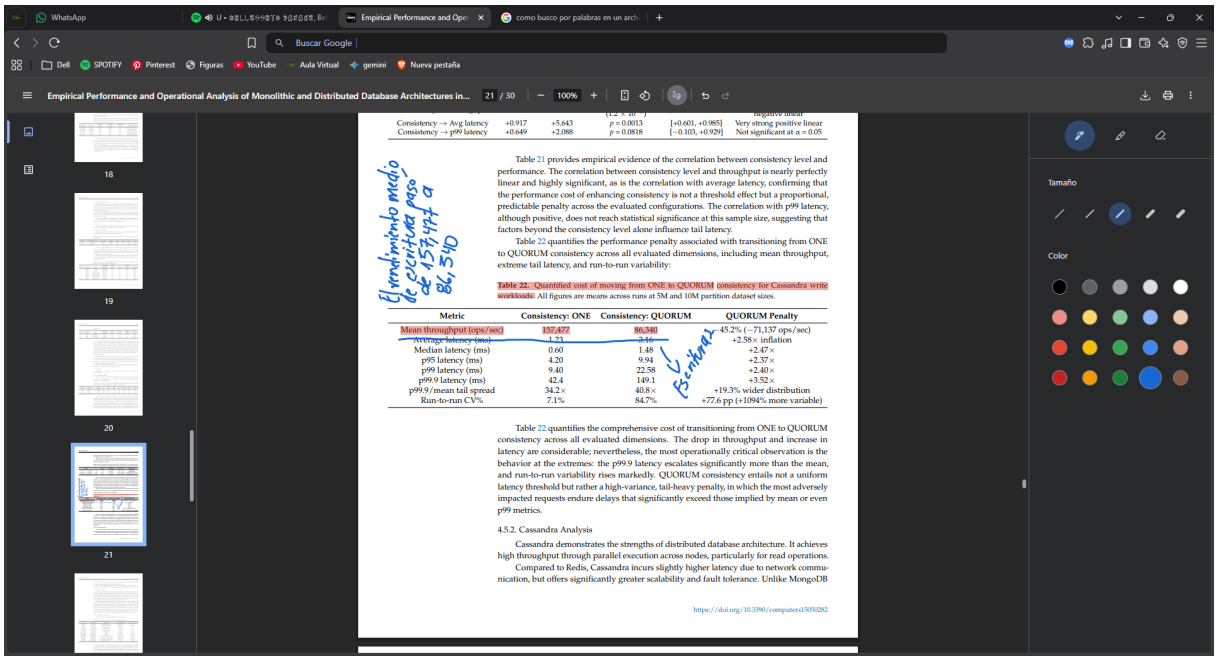


Figura 9: Redžepagić et al., p. 21 del PDF. Tabla 22 sobre el rendimiento de escritura con `ONE` y `QUORUM` [8].